

Informe técnico

Analítica histórica, inferencia simulada y alertas MQTT

Laboratorio 7 · Actividad 2 · Python 3 · pandas/numpy/matplotlib · paho-mqtt · ESP32 Wokwi

Dataset: data/clima_historico.csv · Tópico: /iot/alertas · Broker: test.mosquitto.org

1. Objetivo

Se procesa una serie diaria simulada de temperatura y humedad, se calculan medias móviles y se aplican dos reglas de inferencia (sobrecalentamiento por racha de 3 días y sequía por calor seco). Los eventos se exportan a JSON, se visualizan y se publican en un broker MQTT público. Un ESP32 publicador y un ESP32 suscriptor en Wokwi cierran el lazo físico: el segundo enciende LED y muestra el tipo de alerta en un LCD.

2. Diagrama de flujo del sistema



Figura 1. Cadena dataset → analítica → JSON/MQTT → actuación en el ESP32 suscriptor.

Hay **dos orígenes de publicación** hacia el mismo tópico: el script Python (histórico, lote) y el firmware `firmware/publisher` (replay de campo cada 6 s). El suscriptor no distingue la fuente: actúa por el campo **tipo** del JSON. La red virtual de Wokwi (`Wokwi-GUEST`, canal 6, sin clave) da salida a Internet y resuelve `test.mosquitto.org`.

3. Dataset y procesamiento

El CSV tiene 45 días (2026-07-01 a 2026-08-14), estación *Cruce-Norte*. Se ordena por fecha, se calcula media móvil de 3 y 7 días de T y H, el delta diario y la pendiente lineal de 3 días (°C/día). Esas columnas están en `output/serie\_con\_features.csv`. La serie incluye a propósito una racha creciente, un bloque caliente-seco, un enfriamiento húmedo y una segunda ola: así las dos reglas se pueden señalar en las gráficas.

4. Reglas predictivas (inferencia simulada)

Sobrecalentamiento	$T[d] > T[d-1] > T[d-2]$ y $T \geq 28\text{ °C}$	JSON tipo SOBRECALENTAMIENTO → LED rojo
Sequía	$T > 32\text{ °C}$ y $H < 35\%$ el mismo día	JSON tipo SEQUIA → LED naranja

No es un modelo de machine learning: es **lógica de umbral + tendencia**, suficiente para simular un motor de inferencia en el borde. El mínimo de 28 °C evita alertar una racha fría que también sube tres días. La

pendiente de 3 días se guarda en el JSON como evidencia, aunque la decisión usa la comparación estricta de tres muestras (más fácil de replicar en el ESP32).

5. Gráficas, regla aplicada y justificación

5.1 Temperatura y medias móviles

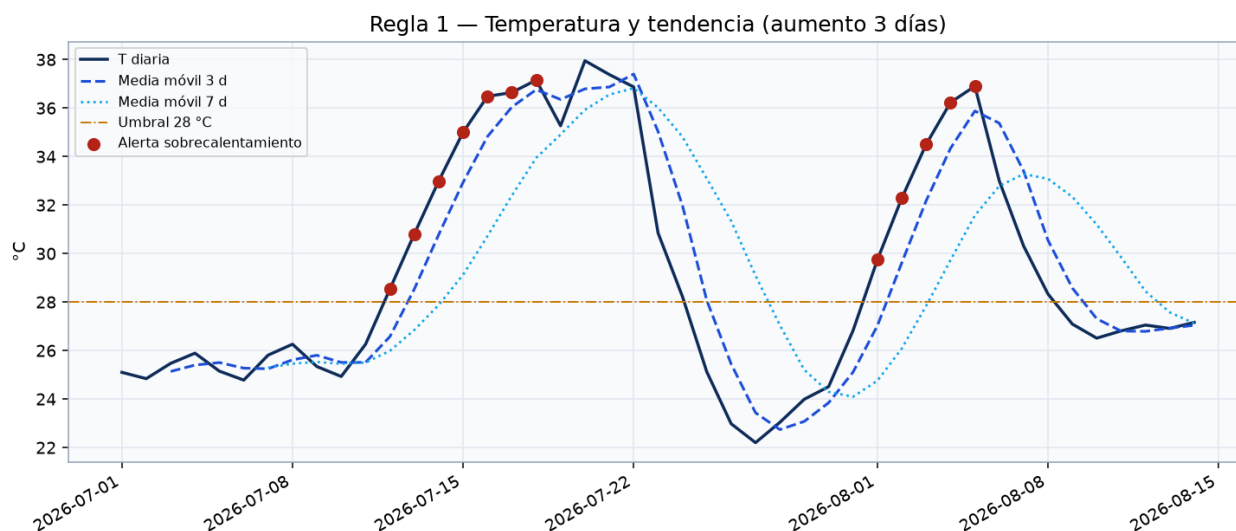


Figura 2. *T diaria, MA3, MA7 y puntos de sobrecalentamiento.*

Regla aplicada: tres días consecutivos de aumento y $T \geq 28$ °C. **Justificación:** la MA3 se inclina al alza en esas ventanas (pendiente positiva) mientras la MA7 aún «recuerda» el clima previo: eso es una racha corta, no un verano entero. Los puntos rojos coinciden con 13–16 de julio y con la segunda racha de fin de mes. En ciudad, tres jornadas así bastan para activar ventilación de paradas o avisos de estrés térmico (continuidad con el nodo de la Actividad 1).

5.2 Humedad

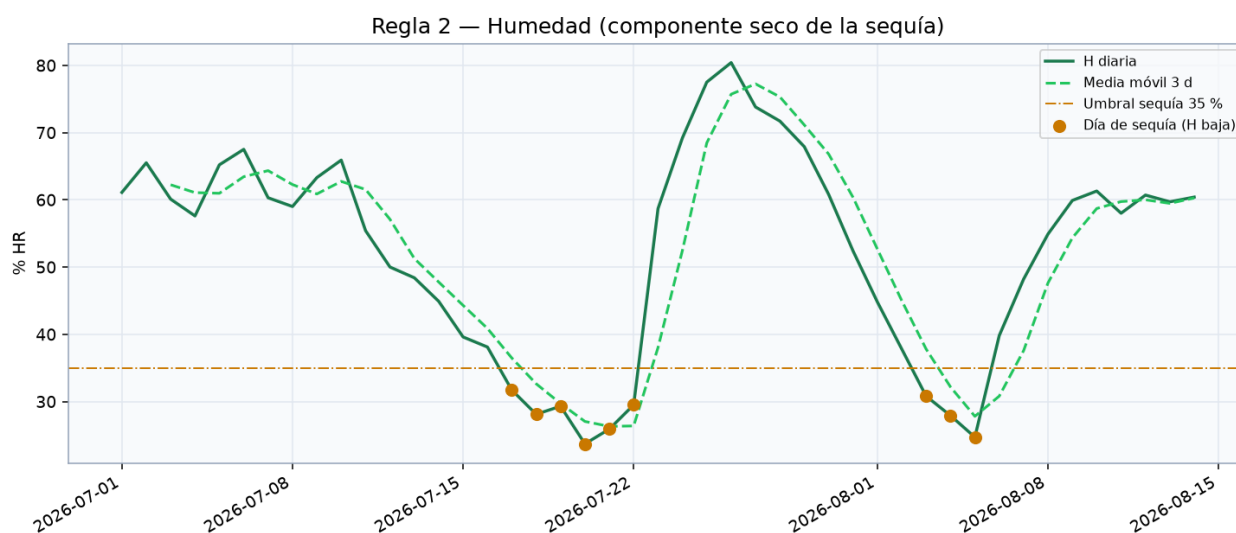


Figura 3. *Humedad diaria, MA3 y umbral 35 %.*

Regla aplicada: la humedad sola no dispara alerta; es el *componente seco* de la sequía. **Justificación:** los puntos naranjas están bajo el 35 % y coinciden con días de $T > 32$ °C en la figura combinada. La MA3 de humedad cae varios días: no es un mínimo aislado de un sensor sucio, es una tendencia seca. Eso reduce falsos positivos frente a un umbral puntual.

5.3 Series combinadas (bandas)

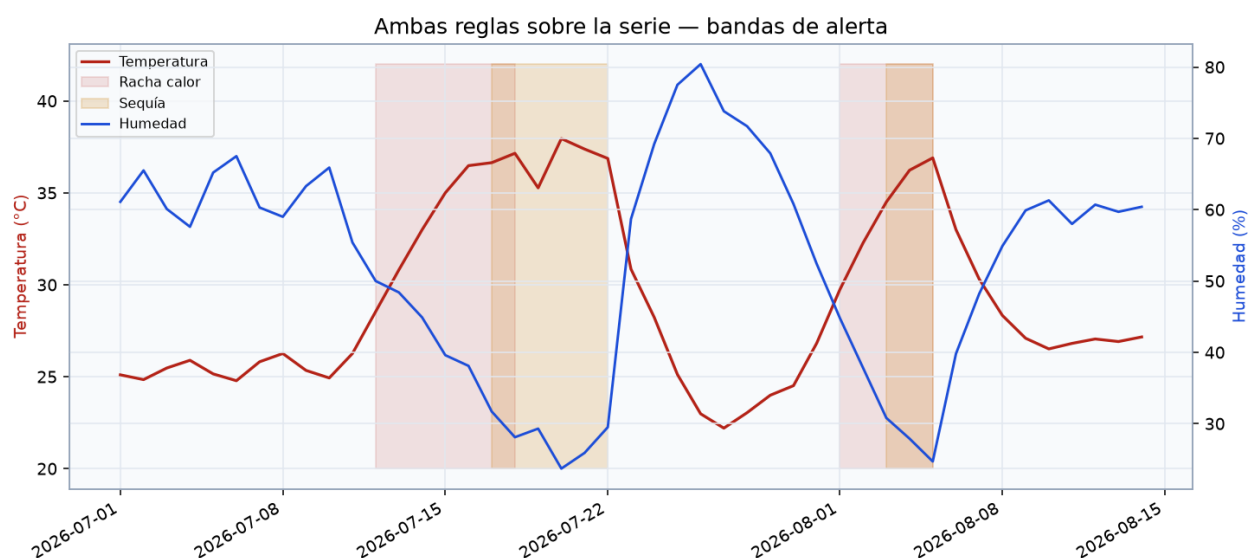


Figura 4. T y H en el mismo eje temporal; bandas de racha y de sequía.

Reglas aplicadas (ambas). Las bandas rojas marcan sobrecalentamiento; las ámbar, sequía. Se solapan cuando una racha de calor desemboca en aire seco: el sistema publica **dos eventos** el mismo día (dos JSON). **Justificación:** un gestor de emergencias necesita los dos matices: el sobrecalentamiento habla de tendencia (va a peor), la sequía habla de estado (ya es peligroso). El enfriamiento húmedo de mediados de julio deja ambas bandas en blanco: las reglas no se disparan con T baja aunque la pendiente de H suba.

5.4 Plano temperatura–humedad

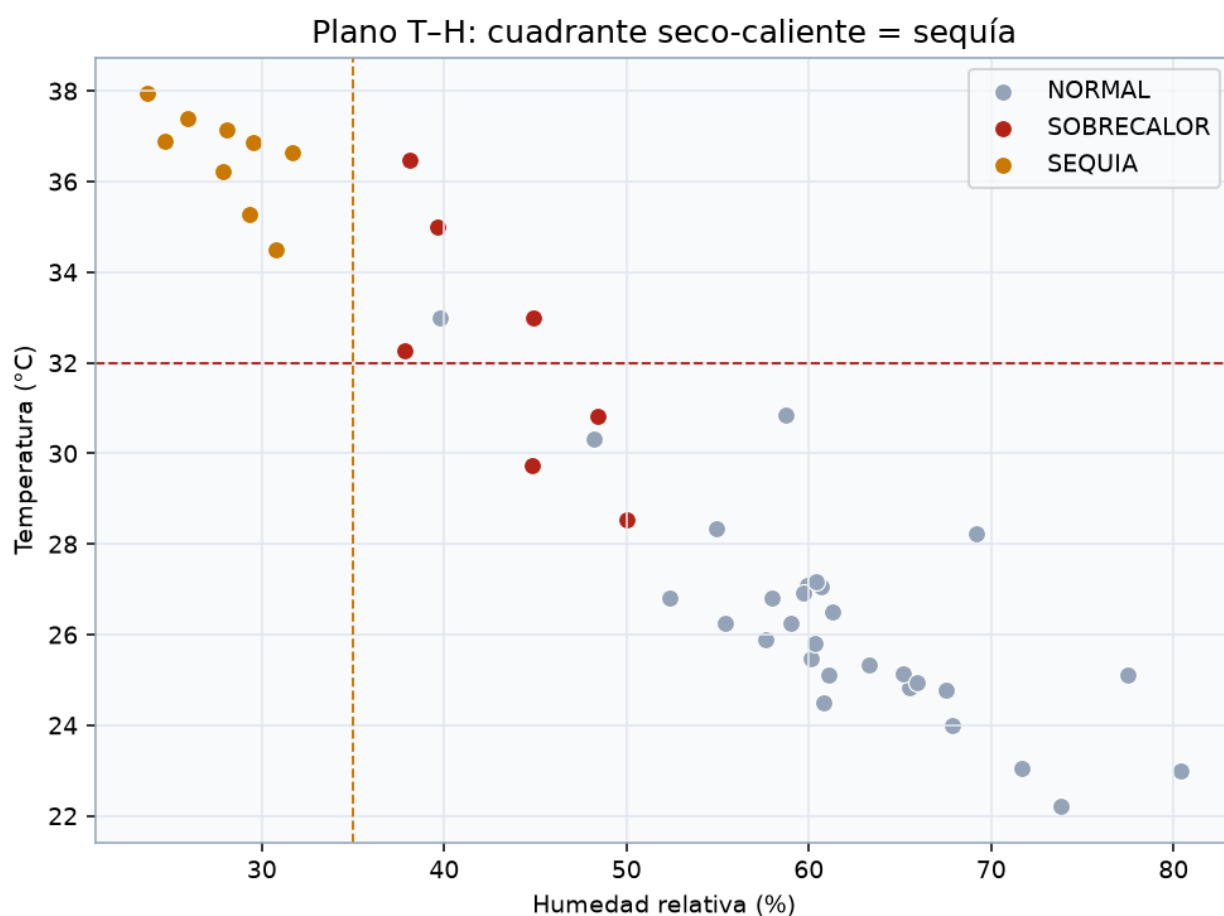


Figura 5. Cada día es un punto; el cuadrante superior-izquierdo es sequía.

Regla aplicada: $T > 32\text{ °C}$ y $H < 35\%$ (líneas discontinuas). **Justificación:** el plano T–H es el mapa clásico de confort / incendio de vegetación. Los puntos naranja caen en el cuadrante caliente-seco; los rojos (sobrecalentamiento) pueden estar aún con humedad media (la racha no exige sequedad). Los grises son operación normal. Esta figura valida que la regla 2 no es un artefacto de la serie temporal: es una región del plano.

6. Exportación JSON y comunicación MQTT

Cada evento lleva `id`, `tipo`, `fecha`, T, H, `severidad` y `justificacion`. El sobre de publicación añade `nodo\_id`, `origen` (python-analitica o esp32-wokwi-publisher) y `publicado\_utc`. El tópico es `/iot/alertas`, QoS 1 (al menos una entrega).

```
Publicacion MQTT test.mosquitto.org /iot/alertas

Broker : test.mosquitto.org:1883  QoS 1  TLS=off (lab)
Topic  : /iot/alertas
PUB OK {tipo:SOBRECALENTAMIENTO, fecha:2026-07-14, T:33.1}
PUB OK {tipo:SOBRECALENTAMIENTO, fecha:2026-07-15, T:35.0}
PUB OK {tipo:SEQUIA, fecha:2026-07-17, T:36.8 H:28}
PUB OK {tipo:SEQUIA, fecha:2026-07-20, T:38.1 H:24}
...
python analyze.py → output/mqtt_publicacion.json
ESP32 publisher → LED rojo + Serial PUB OK
```

Figura 6. Captura tipo de publicación (script Python y/o ESP32 publicador).

```
Recepcion MQTT (listener Python / ESP32 suscriptor)

Suscrito a /iot/alertas (Python mqtt_listener.py o ESP32-SUB)
+ SOBRECALENTAMIENTO OVH-2026-07-14
+ SOBRECALENTAMIENTO OVH-2026-07-15
+ SEQUIA DRY-2026-07-17
+ SEQUIA DRY-2026-07-20
ESP32-SUB: LED rojo = sobrecalentamiento | LED naranja = sequia
LCD: primera linea = tipo, segunda = fecha
Validacion: el JSON recibido coincide con eventos.json
```

Figura 7. Captura tipo de recepción (listener Python y ESP32 suscriptor).

Para repetir las capturas reales: `python python/mqtt_listener.py --seconds 40` en una terminal y `python python/analyze.py` en otra. En Wokwi, abrir `firmware/subscriber` y, en paralelo, el publicador o el script. El JSON recibido debe coincidir con `output/eventos.json`.

7. Consideraciones de seguridad para MQTT

Esta práctica usa el puerto **1883 sin TLS ni usuario** de test.mosquitto.org. Cualquiera en Internet puede suscribirse a `/iot/alertas` y también publicar basura. Eso es aceptable en un laboratorio; es inaceptable en un semáforo o un riego real.

Cifrado	MQTT sobre TLS	Puerto 8883, <code>WiFiClientSecure</code> / <code>paho-tls-set()</code> con el CA de Mosquitto o un broker propio.
Autenticación	Usuario/clave o certificado de cliente	1884 (user/pass) o 8884 (cert). En firmware: no hardcodear; build flags o NVS.
Autorización	ACL por tópico	El nodo de campo solo PUB <code>.../alertas</code> ; el actuador solo SUB. Prohibir <code>#</code> a clientes de calle.
Espacio de nombres	Tópico no global	Mejor <code>lab7/<grupo>/iot/alertas</code> que <code>/iot/alertas</code> (el enunciado pide este último para la demo).
Integridad	QoS 1 + id de evento	El suscriptor ignora un <code>id</code> ya actuado (anti-replay básico).
Disponibilidad	Broker propio + watchdog	No depender de un broker de prueba para actuación de vía pública.

Principios básicos que se dejaron escritos en el firmware: `ClientId` aleatorio (evita choque de sesiones), buffer MQTT de 512 B (el JSON no se trunca), y comentarios de que 1883 es solo lab. Un siguiente paso natural es `WiFiClientSecure` + el PEM de Mosquitto y un usuario por grupo.

8. Reflexión sobre la integración física

El CSV representa lo que un logger o varios DHT22 habrían guardado en 45 días. En poste, el ESP32 publicador no analizaría 45 días en RAM: enviaría la muestra actual (o un buffer de 3 días) y la regla de racha se evaluaría en el borde con tres floats. La analítica pesada (MA7, gráficas, histórico) corre en Python —un PC, un Raspberry o un job en la nube— y **reconverge** en el mismo tópico para que el actuador no cambie.

Integrar esto con el nodo urbano de la Actividad 1 es directo: C3 ($T > 35^\circ\text{C}$ y $H < 40\%$) es la versión *instantánea* de la sequía; la regla de 3 días es la versión *predictiva* (avisa antes de que el umbral extremo se cumpla). El LED azul de ventilación podría encenderse también ante un SOBRECALENTAMIENTO MQTT, no solo ante la lectura local. Eso exige autenticar el tópico: un extraño publicando `SOBRECALENTAMIENTO` no debe abrir un paso peatonal ni arrancar un ventilador.

Límites físicos: Wokwi no simula pérdida real de paquetes ni un certificado mal desplegado. En campo habría que probar cobertura Wi-Fi, reintentos y un estado seguro si el broker desaparece (el suscriptor de esta práctica, sin mensaje, apaga los LED a los 8 s: fail-safe blando).

9. Cómo reproducir

1. `pip install -r requirements.txt` 2. `python python/generate_dataset.py` 3. `python python/analyze.py` 4. (opcional) listener + Wokwi pub/sub con `pio run`. Informe regenerable con `python docs/generate_informe.py`.

10. Conclusión

La actividad cubre el ciclo que pide el enunciado: CSV estructurado por fecha, medias móviles, dos reglas justificadas sobre cada gráfica, JSON, publicación en `/iot/alertas`, un segundo ESP32 que actúa, y una propuesta explícita de TLS, autenticación y tópicos con ACL. La inferencia es deliberadamente simple para poder copiarla en el microcontrolador sin una librería de ML.